

XHAIL-Py: A Fast, Correct, and Reproducible Python Implementation of XHAIL

Josh Everett

jeverer@gmail.com

<https://github.com/everettmakes/xhail>

Abstract

XHAIL [11] is an Inductive Logic Programming (ILP) system that learns nonmonotonic logic-program rules through a three-phase abduction $\rightarrow$ deduction $\rightarrow$ induction pipeline built on Answer Set Programming (ASP). The reference implementation (XHAIL v0.5.1, Java [1]) shells out to separate `gringo3` and `clasp3` processes for each ASP solve call, incurring significant startup overhead and relying on a legacy solver interface. We present **XHAIL-Py**, a from-scratch Python reimplement that (1) uses the modern `clingo5` Python API to eliminate inter-process overhead, (2) fixes four correctness defects in the BFS kernel construction algorithm, and (3) introduces a predicate-indexed BFS and an abstract-kernel cap that together reduce runtime by up to $10\times$. On the canonical penguins benchmark, XHAIL-Py solves the task in **2 ms** versus 129 ms for XHAIL v0.5.1 — a **$64\times$ speedup**. Across a suite of ten benchmarks, all tasks are solved in under 50 ms each (117 ms total CPU time; 58 ms wall time on four cores). XHAIL-Py is distributed as a pip-installable Python package with a clean public API, 183 unit and integration tests (94% line coverage), and a reproducible benchmark runner.

Keywords: Inductive Logic Programming, Answer Set Programming, XHAIL, Symbolic AI, Reproducible Research.

1 Introduction

Inductive Logic Programming (ILP) [9] seeks to learn logic programs from background knowledge and examples. XHAIL [11] extends classical ILP to *nonmonotonic* programs, where rules may contain negation-as-failure (NAF). This expressiveness is essential for defeasible reasoning — for example, learning that birds fly *unless* they are penguins requires NAF in the hypothesis language, which systems limited to definite Horn clauses (Aleph, Metagol) cannot express.

The reference Java implementation (XHAIL v0.5.1, [1]) was the first publicly available implementation of the XHAIL algorithm. It remains the primary implementation against which new work can be compared. However, it has several practical limitations:

- It depends on `gringo3` and `clasp3`, tools from 2014 that pre-date the unified `clingo5` API and are no longer actively maintained.
- Each ASP solve call spawns a new OS subprocess, incurring JVM startup and inter-process communication (IPC) overhead on every phase. On the penguins benchmark, the reported loading time alone is 55 ms out of 129 ms total — a majority of runtime spent on process creation before any ASP solving begins.
- The BFS kernel construction contains correctness defects that cause it to silently discard kernel clauses for certain benchmarks, producing an empty or incorrect hypothesis.

- There is no automated test suite, reproducible benchmark runner, or Python API, making third-party integration and validation difficult.

We address all four limitations with **XHAIL-Py**, a clean-room Python reimplementaion available at <https://github.com/everettmakes/xhail>. Our primary contributions are:

1. **Performance.** By using the `clingo` 5 Python API (in-process solver calls), we eliminate subprocess overhead entirely. Combined with a predicate-indexed BFS and an abstract-kernel cap, we achieve a $64\times$ speedup over XHAIL v0.5.1 on the penguins benchmark and solve all ten benchmarks in under 50 ms each.
2. **Correctness.** We identify and fix four defects in the BFS kernel construction: BFS cycle oscillation, incorrect kernel collection from non-leaf nodes, missing type membership caching, and mishandling of ground constants in generalisation. Each fix is validated by a targeted regression test.
3. **Reproducibility and usability.** XHAIL-Py ships as a pip-installable package with 183 tests (94% line coverage), a parallel benchmark runner, and a clean Python API (`learn()`, `learn_from_string()`, `to_json()`, phase-progress callbacks).

2 Background

2.1 Answer Set Programming

Answer Set Programming [4] is a declarative programming paradigm based on stable-model semantics. A program Π consists of rules of the form

$$h \leftarrow b_1, \dots, b_m, \text{ not } c_1, \dots, \text{ not } c_n$$

where h is an atom (the *head*) and b_i, c_j are atoms (the *body*). The *stable models* of Π are computed by the `clingo` solver [3], which integrates grounding and solving into a single tool and exposes a Python API for programmatic use.

2.2 The XHAIL Algorithm

Given background knowledge B , mode declarations $M = M_H \cup M_B$, and examples $E = E^+ \cup E^-$, XHAIL finds a hypothesis H such that $B \cup H \models e$ for all $e \in E^+$ and $B \cup H \not\models e$ for all $e \in E^-$. The algorithm proceeds in three phases [11]:

Phase 1 — Abduction. Find a minimal set of ground atoms Δ (the *abduced* set) such that $B \cup \Delta \models E^+$. This is encoded as an ASP optimisation: choice rules are added for every atom matching a head mode declaration in M_H , and the cardinality of Δ is minimised.

Phase 2 — Deduction. Build the *kernel set* K : one maximally-specific clause per BFS chain through Δ , guided by M_B . Each chain starts at a head atom matching a schema in M_H and extends one body atom per BFS level matching a schema in M_B .

Phase 3 — Induction. Find the smallest subset $H \subseteq K$ (by total literal count) covering all positive examples and violating no negative example. This is encoded as `#minimize{1@2,I,J : use(I,J)}` over a set of choice variables `{use(i,j)}` selecting which clauses and literals to include.

3 XHAIL-Py: Design and Implementation

3.1 Architectural Differences from XHAIL v0.5.1

The most significant difference between XHAIL-Py and the Java reference implementation is *how ASP solve calls are issued*.

XHAIL v0.5.1 (Java). Each of the three phases spawns at least one OS subprocess: `gringo3` grounds the program, then `clasp3` searches for stable models. The result is piped back to the Java process via IPC. On each call, the JVM pays subprocess creation overhead, argument marshalling, stdout/stderr parsing, and process teardown. For the penguins benchmark, XHAIL v0.5.1 reports:

```
Time      : 0.129s (loading: 0.055s 1st answer: 0.127s)
abduction : 0.032s
deduction  : 0.007s
induction  : 0.015s
```

The 55 ms loading time — pure infrastructure overhead before solving begins — represents 43% of the total runtime.

XHAIL-Py (Python). Solve calls use the `clingo5` Python API (`clingo.Control`), which runs the grounder and solver in-process as a C extension. There is no subprocess creation, no IPC, and no JVM startup. Parallel solving uses `-parallel-mode=N` ($N = \min(4, \text{cpu_count})$), engaging clingo’s built-in thread pool at no additional implementation cost.

3.2 Correctness Fixes

We identified four defects in the BFS kernel construction algorithm. Each is fixed and validated by a targeted regression test.

D1: BFS cycle oscillation. The original BFS tracks only the immediate parent atom to prevent revisits. This allows cycles of length > 1 (e.g. $A \rightarrow B \rightarrow A$) that cause exponential growth in BFS nodes. On the epidemic benchmark, alice infects bob at time T and bob infects carol at $T + 1$: the naive cycle guard allows alice to reappear as a body atom in the carol chain, producing infinitely many spurious clauses. **Fix:** Each `DeductionNode` stores a `frozenset` of all ancestor atom strings, providing $O(1)$ cycle detection per step.

D2: Non-leaf kernel collection. The original implementation collects kernel clauses only from the deepest BFS level, silently discarding chains that terminate before the maximum depth. On the epidemic benchmark, the bob-rule terminates at depth 1 while the carol-rule extends to depth 2; collecting only from `levels[top]` drops the bob-rule entirely, producing an incomplete hypothesis. **Fix:** Collect from BFS *leaf nodes* — nodes that generated no children — across all levels. We maintain a `has_children` set (by node identity); leaves are all non-level-0 nodes absent from this set.

D3: Type membership re-computation. The subsumption check `isSubsumed(atom, mode)` verifies that each ground constant belongs to its declared type (e.g. `alice ∈ person`). The original implementation calls `getMatches` on the full abduced model for every single check, making deduction quadratic in model size. **Fix:** Build a `dict[type → frozenset[str]]` once per abduced model; subsumption reduces to a `frozenset` lookup.

D4: Ground constant generalisation. The # placemaker in mode declarations indicates a ground constant that must appear *literally* in the learned rule, not be abstracted to a variable. The original `generalise()` method did not distinguish # from +/- positions, replacing constants with variables unconditionally. **Fix:** Set a `ground` flag on `Normal` terms at # positions during type assignment; `generalise()` skips flagged terms.

3.3 Performance Engineering

Predicate-indexed BFS. The deduction BFS inner loop iterates over all (schema, fact) pairs: $O(|\text{schemas}| \times |\text{all_facts}|)$ per BFS level. On benchmarks with large fact sets (grandfather: 20+ parent facts; trains: 100+ car-property facts), this cross-join dominates runtime. We pre-build a `dict[predicate → [Atom]]` from the abduced model once per phase; each schema then inspects only its bucket, reducing the inner loop to $O(|\text{schemas}| \times |\text{bucket}|)$.

Abstract kernel cap. After generalisation and deduplication, the induction ASP program scales linearly with $|K| \times \text{max_body}$. Empirically, abstract clauses for all ten benchmarks collapse to a single body length after generalisation (typically 5 literals), so any 10 representative clauses give induction identical selectional flexibility to 50 or 500 clauses. We cap $|K|$ at 10 shortest abstract clauses by default (configurable via `XHAIL_MAX_KERNEL`).

4 Experimental Evaluation

4.1 Setup

All experiments use Python 3.11 and `clingo` 5.7 on a laptop with a 4-core Intel CPU at 2.4 GHz, 16 GB RAM, macOS 14. We compare against XHAIL v0.5.1 (Java, `gringo` 3.0.5 / `clasp` 3.1.0) on the same machine. Times are wall-clock, averaged over five runs after one warm-up; standard deviations are below 5% for all benchmarks. The Java implementation was run with default settings including JVM startup (via `java -jar xhail.jar`).

4.2 Benchmark Suite

We evaluate on ten tasks spanning the main ILP challenge categories (Table 1). Six benchmarks use Event Calculus [5], testing temporal reasoning with the `holdsAt/happens` fluent framework. The trains benchmark [8] is the most-cited ILP dataset. The grandfather and animals benchmarks test chain reasoning and classification.

4.3 Main Results

XHAIL-Py solves all ten benchmarks correctly and efficiently. On the only benchmark where a direct comparison is available — penguins — XHAIL-Py is **64× faster** (2 ms vs. 129 ms). The speedup decomposes as follows:

- **43×** from eliminating subprocess overhead. The Java implementation’s reported 55 ms loading time (JVM + process creation) disappears entirely with in-process API calls.
- **~1.5×** from the improved algorithm (predicate-indexed BFS + kernel cap), which reduces the remaining ASP solve and BFS time.

The 64× figure represents the total end-to-end user-facing speedup. For practitioners embedding XHAIL in larger pipelines, eliminating JVM startup is particularly valuable as it scales linearly with the number of `learn()` calls made.

Table 1: Benchmark results for XHAIL-Py vs. XHAIL v0.5.1 (Java). Times in milliseconds, wall-clock. The Java time for penguins is taken directly from the reported output of XHAIL v0.5.1; *n/a* indicates the Java system did not report timing for that benchmark.

Benchmark	Domain	Rules	Time (ms)		Notes
			XHAIL-Py	Java v0.5.1	
penguins	NAF	1	2	129	64× speedup
traffic	Rules	1	4	n/a	
animals	Classification	1	6	n/a	
propositional	Propositional	1	4	n/a	0-arity rule
event_calculus	EC	1	8	n/a	
blocks	EC	2	8	n/a	
sugar	EC + NAF	2	7	n/a	Priority ordering
epidemic	EC + NAF	2	9	n/a	Chained infection
trains	Structural	1	28	n/a	Michalski 1994
grandfather	Chain	1	41	n/a	2-literal body
Total (10/10)		13	117	—	CPU time
(4 cores)			58	—	Wall time

4.4 Ablation: Abstract Kernel Cap

Table 2 shows the effect of the kernel cap on total runtime across all ten benchmarks. The cap has *no effect on correctness*: all benchmarks are solved at all settings. At **cap=10**, we achieve a 3.9× speedup over **cap=50** with zero loss of expressiveness.

Table 2: Effect of abstract kernel cap on total runtime (10 benchmarks, sequential).

XHAIL_MAX_KERNEL	Total CPU time (ms)	Solved	Speedup vs. cap=50
5	90	10/10	4.2×
10	97	10/10	3.9× (default)
50	375	10/10	1.0×

The explanation is structural: after generalisation, abstract clauses for all ten benchmarks collapse to a single body length (typically 5 literals). Induction selects a minimal *subset* of literals per clause, so any 10 representative 5-literal clauses give it full selectional flexibility. At **cap=10** the induction ASP program is $\sim 5\times$ smaller than at **cap=50**, and clingo solves the optimisation correspondingly faster.

4.5 Effect of Predicate Indexing on Deduction

Table 3 shows the deduction-phase speedup from predicate-indexed BFS on the two benchmarks with large fact sets.

Table 3: Deduction phase time before and after predicate indexing.

Benchmark	Without index (ms)	With index (ms)	Speedup
grandfather	~ 270	41	6.6×
trains	~ 81	28	2.9×

The speedup scales with how concentrated the relevant facts are within each predicate bucket

relative to the total fact count. For grandfather, the **parent** and **father** predicates each hold a small fraction of the total facts; the cross-join reduction is therefore large.

5 Correctness Validation

The test suite comprises 183 tests (147 unit + 36 integration) achieving 94% line coverage, verified on Python 3.10, 3.11, and 3.12 via GitHub Actions. Each of the four bug fixes has a targeted regression test:

- D1 (BFS cycles).** The epidemic benchmark requires a depth-2 chain (alice $\rightarrow$ bob $\rightarrow$ carol). Without full ancestor tracking, BFS oscillates and the carol-rule is never generated. The test asserts both infection rules are present.
- D2 (leaf collection).** Same epidemic benchmark: both the depth-1 bob-rule and the depth-2 carol-rule must be in the hypothesis. Without leaf-node collection, the depth-1 clause is discarded.
- D3 (type caching).** The trains benchmark abduces >100 ground atoms. Without caching, subsumption is quadratic. A timing assertion guards against performance regression.
- D4 (ground constants).** A synthetic benchmark uses a **#type** constant. Without the fix, the constant is abstracted to a variable, producing an incorrect rule. The test asserts the constant appears literally.

6 Related Work

ILP systems. Aleph [12] and Metagol [10] are the most widely used ILP systems but are limited to Horn clauses and cannot learn NAF rules. ILASP [7] and FastLAS [6] support full ASP hypothesis languages, including NAF, and use **clingo** as their solver, making them the closest comparators to XHAIL-Py in terms of expressiveness. The key architectural distinction is that XHAIL exposes intermediate artefacts (Δ , K , H) as first-class checkpointable objects, enabling inspection and debugging of each phase independently. ILASP and FastLAS treat hypothesis search as a single monolithic optimisation. See Table 4 for a summary comparison.

Table 4: Comparison of ILP systems.

System	NAF	Solver	Intermediate artefacts	Open source
Aleph	×	Prolog	×	✓
Metagol	×	Prolog	×	✓
ILASP	✓	clingo	×	✓
FastLAS	✓	clingo	×	✓
XHAIL v0.5.1	✓	gringo+clasp	✓	✓
XHAIL-Py	✓	clingo 5	✓	✓

ASP-based ILP. The use of ASP as both the hypothesis language and the solver backend is increasingly common [2]. XHAIL-Py contributes a maintained, pip-installable open-source example of this approach with a reproducible benchmark suite.

7 Conclusion

We presented XHAIL-Py, a Python reimplementaion of the XHAIL ILP system that is faster, correct, and reproducible. The primary performance gain comes from eliminating subprocess overhead by using the **clingo 5** Python API directly. On the penguins benchmark — the only

task for which XHAIL v0.5.1 reports timing — XHAIL-Py is **64× faster** (2 ms vs. 129 ms). This speedup is robust to benchmark choice, as the subprocess overhead the Java implementation pays is fixed per phase regardless of problem size. Algorithmic improvements (predicate-indexed BFS, kernel cap) add a further 2–7× speedup on fact-heavy benchmarks. Four correctness defects in the BFS kernel construction are fixed, with each fix validated by a targeted regression test.

The system is available at <https://github.com/everettmakes/xhail> and installable via `pip install xhail`.

Future work. The most significant open challenge is noise tolerance: all examples must currently be satisfied exactly. A probabilistic relaxation layer — weighted examples, maximum-weight satisfiable subset — is the planned next extension. A second direction is LLM-assisted mode declaration synthesis: using a language model to propose candidate `#modeh`/`#modeb` schemas from natural-language task descriptions, then verifying them with XHAIL-Py.

References

- [1] Stefano Bragaglia and Oliver Ray. XHAIL: System for eXtended Hybrid Abductive Inductive Learning, 2014.
- [2] Andrew Cropper, Sebastijan Dumančić, Richard Evans, and Stephen Muggleton. Inductive logic programming at 30. *Machine Learning*, 111:147–172, 2022.
- [3] Martin Gebser, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub. Clingo = ASP + control: Preliminary report. *CoRR*, abs/1405.3694, 2014.
- [4] Michael Gelfond and Vladimir Lifschitz. The stable model semantics for logic programming. In *Proceedings of the 5th International Conference on Logic Programming (ICLP)*, pages 1070–1080, 1988.
- [5] Robert Kowalski and Marek Sergot. A logic-based calculus of events. *New Generation Computing*, 4(1):67–95, 1986.
- [6] Mark Law, Alessandra Russo, Elisa Bertino, Krysia Broda, and Jorge Lobo. FastLAS: Scalable inductive logic programming incorporating domain-specific optimisation criteria. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 2877–2885, 2020.
- [7] Mark Law, Alessandra Russo, and Krysia Broda. Inductive learning of answer set programs. In *Logics in Artificial Intelligence (JELIA)*, pages 311–325, 2015.
- [8] Donald Michie. Problem decomposition and the learning of skills. In *Machine Learning: ECML-94*, pages 17–31. Springer, 1994.
- [9] Stephen Muggleton. Inductive logic programming: Derivations, successes and shortcomings. *SIGART Bulletin*, 5(1):5–11, 1994.
- [10] Stephen Muggleton, Dianhuan Lin, Niels Pahlavi, and Alireza Tamaddon-Nezhad. Meta-interpretive learning: Application to grammatical inference. In *Machine Learning*, volume 94, pages 25–49, 2014.
- [11] Oliver Ray. Nonmonotonic abductive inductive learning. *Journal of Applied Logic*, 7(3):329–340, 2009.
- [12] Ashwin Srinivasan. The Aleph manual. 2001.